PHASE 3: SYSTEM DESIGN

Agesta – AI-Generated Smart Tax Assistant (India-Friendly)

Purpose of System Design Phase

System Design converts the requirements identified in Phase 2 into a technical blueprint that explains:

- How the system is structured
- How components interact
- How data is stored
- How responsibilities are divided

This phase answers the question: HOW will the system be built?

Inputs to This Phase

This phase uses outputs from Requirements Analysis (Phase 2):

- User roles (Taxpayer, Admin)
- Functional requirements
- Non-functional requirements

Without requirements analysis, system design cannot be performed correctly.

System Design Deliverables and Items

- ✓ Use case diagram prepared
- ✓ Architecture diagram prepared
- ✓ Database schema designed
- ✓ Module breakdown completed

3.1 Use Case Diagram

Purpose

The use case diagram visually represents:

- System functionality
- Interactions between users and the system

It ensures that all functional requirements are covered.

Input Used

- User roles from Phase 2
- Functional requirements ("The system shall …")

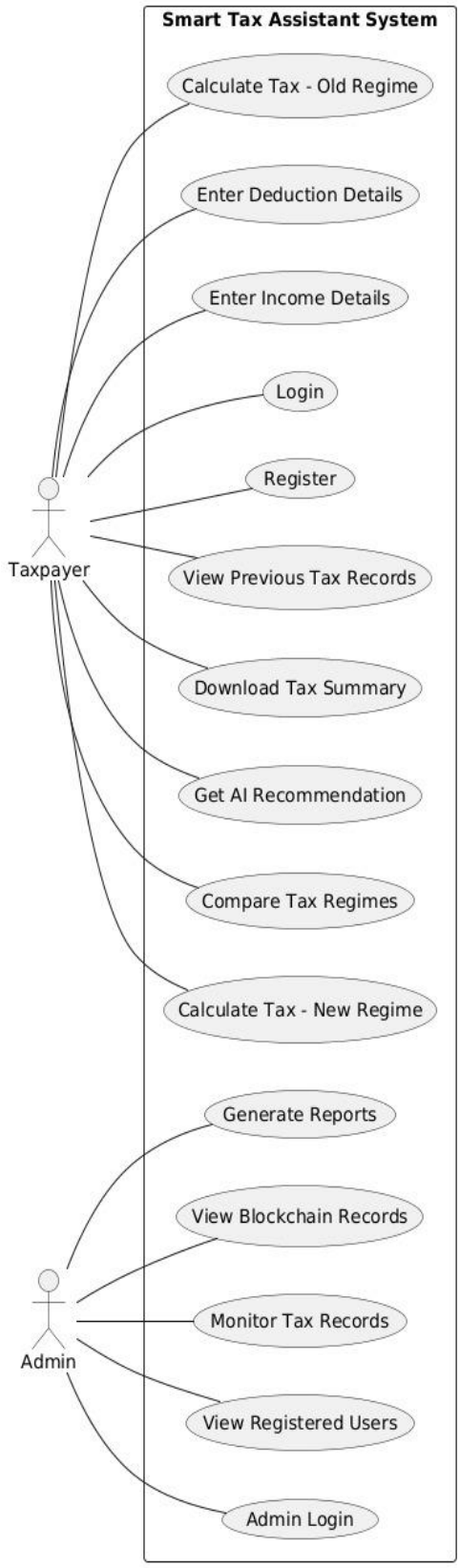
Steps to Create Use Case Diagram

- Identify actors (Taxpayer, Admin)
- Convert each functional requirement into a use case
- Draw system boundary
- Connect actors to their respective use cases

Online Platforms to Generate Use Case Diagram

- Recommended for academics: PlantUML (Reusable, clean)
- Lucidchart
- Draw.io

Agesta Use Case Diagram (With Description)



Actors: Taxpayer, Admin

Taxpayer Use Cases:

- Register/Login
- Input Income and Expense Details
- Apply Deductions (80C, 80D, HRA)
- Calculate Tax (Old Regime)
- Calculate Tax (New Regime)
- View Comparison Results
- Get AI Recommendation
- Download Tax Summary
- View Blockchain-Secured Records

Admin Use Cases:

- Login to Admin Panel
- View All Taxpayers
- Monitor Tax Calculations
- Enable/Disable User Accounts
- Generate System Reports
- Manage Deduction Rules

Output

- Actors clearly defined
- Use cases mapped to requirements
- System boundary established

3.2 Architecture Diagram

Purpose

The architecture diagram explains:

- Overall system structure
- Interaction between layers
- Flow of data and control

Architecture Style Used

Three-Tier (Layered) Architecture

Architecture Layers

Presentation Layer

- Web Interface (Frontend)
- Admin Dashboard

Application / Business Logic Layer

- Authentication Module
- Tax Calculation Engine
- Regime Comparison Module
- AI Recommendation Engine
- Blockchain Integration Module
- Report Generation Module

Data Layer

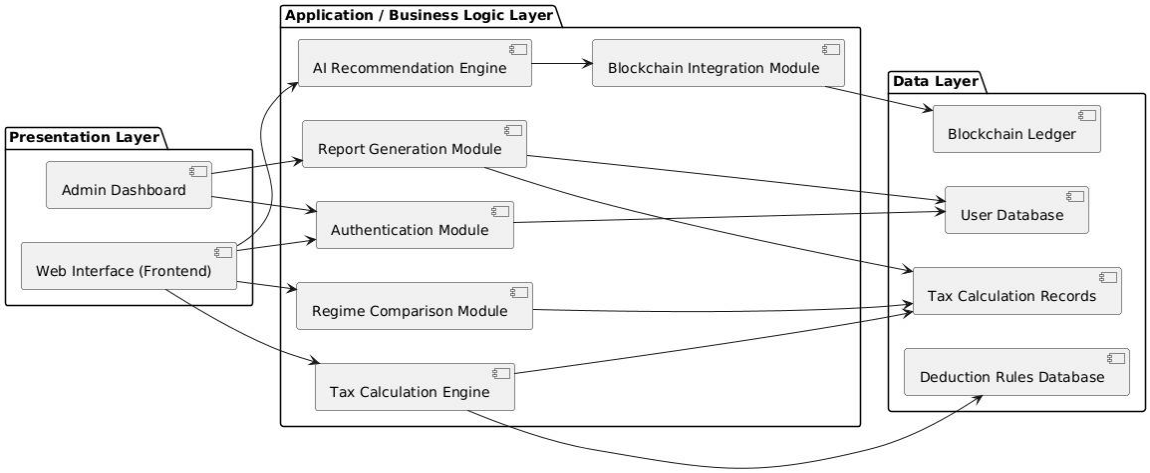
- User Database
- Tax Calculation Records
- Deduction Rules Database
- Blockchain Ledger (for secure storage)

Steps to Create Architecture Diagram

- Group functional requirements into modules
- Place modules into appropriate layers
- Define communication between layers
- Show database and blockchain as separate layers

Architecture Flow (Text Description)

Presentation Layer ↔ Business Logic Layer ↔ Data Layer



1. Taxpayer enters income/expense details via web interface
2. Request sent to Tax Calculation Engine (Business Logic)
3. Engine calculates tax under both regimes using Deduction Rules Database
4. AI Recommendation Engine analyzes results
5. Results stored in User Database & Blockchain
6. Response sent back to Presentation Layer for display

Output

- Clear separation of concerns
- Modular and scalable design
- Easy future enhancements

3.3 Database Schema Design

Purpose of Database Schema

The database schema defines:

- What data is stored
- How data is structured
- How different system modules share data
- How system integrity is maintained

The schema is directly derived from functional requirements, not from assumptions.

Design Approach Used

Logical Schema Design (Recommended for Mini Projects)

Instead of complex ER diagrams:

- We define collections/tables
- Specify key attributes
- Explain relationships in words

This approach is:

- Time-efficient
- Implementation-ready
- Accepted in academic evaluations

Core Design Principles

- Single source of truth
- Avoid data duplication
- Role-based separation (Taxpayer, Admin)
- Tax calculation lifecycle tracking

Identified Entities (From Functional Requirements)

From Phase 2, the system needs to store:

- Users (Taxpayer, Admin)
- Taxpayer Income Details
- Tax Deductions
- Tax Calculations (Old & New Regime)
- Tax Comparison Results
- AI Recommendations
- Tax Summary Reports
- Blockchain Records

Detailed Logical Database Schema

Entity / Table	Key Attributes
1. Users	user_id, email, password_hash, full_name, pan, mobile, role (Taxpayer/Admin), registration_date, status (Active/Inactive)
2. Income_Details	income_id, user_id (FK), salary, freelance_income, business_income, other_income, financial_year
3. Tax_Deductions	deduction_id, user_id (FK), deduction_80c, deduction_80d, hra_exemption, standard_deduction, other_deductions
4. Tax_Calculations	calculation_id, user_id (FK), regime_type (Old/New), taxable_income, total_tax, tax_rate, calculation_date
5. Tax_Comparison	comparison_id, user_id (FK), old_regime_tax, new_regime_tax, tax_difference, recommended_regime, comparison_date
6. AI_Recommendations	recommendation_id, user_id (FK), comparison_id (FK), recommended_regime, confidence_score, explanation_text
7. Tax_Summary_Reports	report_id, user_id (FK), financial_year, summary_pdf, total_income, total_tax, recommendation, generated_date
8. Blockchain_Records	blockchain_id, user_id (FK), report_id (FK), block_hash, timestamp, stored_on_blockchain (Yes/No)

Entity Descriptions

1. Users Collection / Table

Stores all system users, differentiated by role.

 Why single Users table? Reduces duplication and simplifies authentication.

2. Income_Details Collection

Stores taxpayer income from all sources (salary, freelance, business, other).

 Keeps income logic separate from deductions and calculations.

3. Tax_Deductions Collection

Stores deduction amounts claimed by taxpayer (80C, 80D, HRA).

 Only applicable for Old Regime calculations.

4. Tax_Calculations Collection

Stores calculated tax for both Old and New regimes separately.
 Central table showing actual tax liability.

5. Tax_Comparison Collection

Compares tax liability between two regimes.
 Shows difference and highlights which regime is beneficial.

6. AI_Recommendations Collection

Stores AI-generated recommendation with explanation.
 Provides confidence score and reasoning for recommendation.

7. Tax_Summary_Reports Collection

Stores generated tax summary PDF and metadata.
 Used for download and record keeping.

8. Blockchain_Records Collection

Stores blockchain hash and timestamp for secure storage.
 Ensures tamper-proof record of tax summaries.

Relationships Explained (Textual)

 No complex joins required for mini project.

Relationship	Description
User → Income_Details	One user has many income records (One-to-Many)
User → Tax_Deductions	One user claims deductions for specific FY (One-to-Many)
User → Tax_Calculations	One user has calculations for both regimes (One-to-Many)
Tax_Calculations → Tax_Comparison	Two calculations (Old+New) linked to one comparison
Tax_Comparison → AI_Recommendations	One comparison generates one AI recommendation
User → Tax_Summary_Reports	One user has multiple reports (One-to-Many)
Tax_Summary_Reports → Blockchain_Records	Each report stored as blockchain record (One-to-One)

MongoDB vs SQL Mapping (For Students)

MongoDB (NoSQL/MERN)	SQL (Relational)
users	users
incomeDetails	income_details
taxDeductions	tax_deductions
taxCalculations	tax_calculations

taxComparison	tax_comparison
aiRecommendations	ai_recommendations
taxSummaryReports	tax_summary_reports
blockchainRecords	blockchain_records

 Schema works for both paradigms.

Why This Schema Is Academically Strong

- ✓ Derived from functional requirements
- ✓ Normalized but simple
- ✓ Directly implementable
- ✓ Easy to explain in viva
- ✓ Supports AI and Blockchain integration

3.4 Module Breakdown

What is a Module?

A module is a logical part of the system that performs a specific set of responsibilities. Each module focuses on one concern only, which makes the system:

- Easier to understand
- Easier to develop
- Easier to test and maintain

1. Authentication Module

Purpose:

This module controls who can access the system.

Responsibilities:

- Register new users (Taxpayer / Admin)
- Authenticate users during login
- Identify user role after login
- Reset password functionality

Why This Module is Important:

- Prevents unauthorized access
- Ensures role-based access (Admin ≠ Taxpayer)
- Used by all other modules

2. Income Input Module

Purpose:

Captures taxpayer income from multiple sources.

Responsibilities:

- Accept salary income
- Accept freelance/business income
- Accept other sources of income
- Validate income data
- Store income data securely

Why This Module is Important:

- Directly solves the problem (accept user details)
- Foundation for tax calculation
- Ensures data integrity

3. Deduction Management Module

Purpose:

Manages tax deductions applicable only in Old Regime.

Responsibilities:

- Accept 80C deductions (insurance, investments)
- Accept 80D deductions (health insurance)
- Calculate HRA exemption
- Apply standard deduction
- Validate deduction amounts

Why This Module is Important:

- Old Regime depends on deductions
- Key differentiator between Old and New regimes
- Enables accurate Old Regime calculation

4. Tax Calculation Engine Module

Purpose:

Calculates tax under both Old and New regimes.

Responsibilities:

- Calculate taxable income (Old Regime)
- Apply deductions and exemptions
- Calculate tax based on applicable tax brackets
- Calculate taxable income (New Regime)
- Apply lower tax rates

- Return final tax liability

Why This Module is Important:

- Core functionality of the system
- Accuracy critical for taxpayers
- Handles complex tax bracket logic

5. Comparison & Analysis Module

Purpose:

Compares tax liability between regimes.

Responsibilities:

- Compare Old vs New regime tax
- Calculate tax savings difference
- Identify which regime is beneficial
- Generate comparison reports

Why This Module is Important:

- Directly solves the main problem (comparison)
- Enables informed decision-making
- Central to system value proposition

6. AI Recommendation Engine Module

Purpose:

Provides AI-based recommendation using comparison results.

Responsibilities:

- Analyze tax comparison data
- Apply AI/ML algorithms to find better regime
- Generate confidence score
- Provide explanation in simple language

Why This Module is Important:

- Adds AI intelligence to the system
- Simplifies complex decision for taxpayers
- Builds trust through explanation

7. Report Generation Module

Purpose:

Generates downloadable tax summary reports.

Responsibilities:

- Format tax summary in user-friendly way

- Generate PDF reports
- Include all calculations and recommendations
- Prepare report for download

Why This Module is Important:

- Provides downloadable proof of calculations
- Users can share with tax consultants
- Solves record-keeping problem

8. Blockchain Integration Module

Purpose:

Securely stores tax summaries on blockchain.

Responsibilities:

- Create blockchain transaction for tax summary
- Generate cryptographic hash
- Store hash and timestamp
- Enable verification of record authenticity
- Retrieve blockchain records when needed

Why This Module is Important:

- Solves data integrity and security problem
- Prevents record alteration or loss
- Provides audit trail
- Ensures compliance and transparency

9. Admin Management Module

Purpose:

Provides system control and monitoring for administrators.

Responsibilities:

- View registered taxpayers
- Monitor tax calculations
- Enable/disable user accounts
- View audit logs
- Generate system reports

Why This Module is Important:

- Maintains system integrity
- Helps resolve misuse or errors
- Required for system supervision

10. Notification Module

Purpose:

Keeps users informed about their tax calculations.

Responsibilities:

- Send calculation completion notifications
- Alert users about tax optimization opportunities
- Send download links for reports
- Notify about blockchain storage completion

Why This Module is Important:

- Improves user experience
- Ensures timely awareness of results
- Reduces need for manual follow-up

Final Note (VERY IMPORTANT)

 Module breakdown does not mean coding modules immediately.

 It is a design-level division, not implementation-level.

Each module:

- Maps directly to functional requirements
- Will later become: APIs, Controllers, Services in chosen stack

So, Should You Explain This?

✅YES — Explain:

- Purpose of each module
- Responsibilities
- Why it exists

❌NO — Do NOT explain:

- Code
- Framework-specific logic
- Database queries

PHASE 3 STATUS

✅SYSTEM DESIGN COMPLETED & FROZEN

All deliverables prepared:

- ✓ Use Case Diagram described
- ✓ Three-Tier Architecture designed
- ✓ Logical Database Schema created
- ✓ 10 Modules defined with clear responsibilities